

Заменит ли ИИ-программирование всех программистов? Взгляд на реальное положение дел в отрасли

Zhihu · 2026-09-19

С конца 2025 — начала 2026 года, после того как возможности ИИ-программирования резко выросли, значительную часть работы программистов по написанию кода — а то и почти всю — взял на себя ИИ. Это явление вызвало в среде программистов панику и отчаяние. Одновременно компании, работающие в области ИИ, используя успехи этой сферы, безмерно преувеличивают возможности ИИ, обещая, что через год-два все «белые воротнички» останутся без работы. Бессчётные блогеры и самостийные медиа подхватывают это, распространяя панику и нагоняя тревогу.

К этому вопросу стоит отнестись серьёзно. Способно ли ИИ-программирование заменить всех программистов? Если нет, то какова реальная доля замещения?

Собственно, в одном из своих ответов несколько месяцев назад («Первый удар ИИ — по программистам: неужели программистов ждёт массовая безработица?») я уже высказал своё суждение: ИИ не способен заменить весь человеческий труд, а сквозная доля замещения не превышает 30 %.

Когда в производство в широких масштабах войдут интеллектуальные машинные системы на основе ИИ, ручное кодирование в основном исчезнет. Разумеется, разработка ядра со сложной архитектурой или особо высокими требованиями к производительности, возможно, по-прежнему будет требовать активного участия человека, но массовая обычная разработка кода будет полностью заменена ИИ. Сколько же людей это заменит? Судя по моему собственному опыту использования и отзывам других людей на Zhihu, прирост эффективности составляет от 20 до 50 %; если AI coding действительно внедрится (сейчас в отрасли идёт широкомасштабный эксперимент), то при доле чистых разработчиков в 60 % это сократит примерно от 12 до 30 % программистских рабочих мест.

Массовое внедрение AI coding идёт уже как минимум полгода, особенно в Китае и США, где крупные компании практически в обязательном порядке требуют от сотрудников разрабатывать с помощью ИИ-агентов. Так каков же реальный эффект?

Согласно докладу McKinsey^[1], у программистов, «умеющих пользоваться ИИ», производительность можно повысить примерно на 16–30 % — в целом это совпадает с моей прежней оценкой.

Figure 1

Одно из недавних исследований^[2] также показывает: хотя ИИ-программирование и значительно ускорило генерацию кода, в пересчёте на конечный выпуск продукта эффективность выросла лишь примерно на 30 %. Это вновь совпадает с исследованием McKinsey выше и с моими прежними предположениями.

Инструменты генеративного ИИ для разработки программного обеспечения совершенствуются стремительно: за считанные годы они прошли путь от простого автодополнения кода до полностью автономных кодирующих агентов (coding agents). Превращаются ли эти успехи в рост производительности на уровне отдельных задач в реальном мире? И даже если превращаются, означает ли рост эффективности на уровне задач увеличение конечного выпуска? Эти вопросы критически важны для понимания экономического влияния ИИ. Проведя исследование совпадающих событий по разработчикам, мы обнаружили, что по мере обновления и совершенствования инструментов ИИ-кодирования их эффект на уровне задач становится всё заметнее: функция автодополнения увеличила недельное количество коммитов примерно на 30 %, синхронные агенты (sync agents) подняли этот накопленный эффект примерно до 180 %, а асинхронные агенты (async agents) — примерно до 240 %. Однако при переходе на более высокий уровень — к этапу выпуска — этот прирост резко ослабевает: хотя синхронные агенты увеличили число строк кода на 958 %, а пул-реквесты (pull requests) — на 86 %, количество итоговых выпущенных версий выросло лишь на 20 %. Та же закономерность наблюдается и в общей активности по написанию кода на GitHub.

Figure 2

Источник: <https://mitsloan.mit.edu/ideas-made-to-matter/ai-boosts-worker-productivity-does-translate-to-final-outputs>

Ограниченность ИИ-программирования подтверждают не только исследования: разработчики и руководители, реально применяющие ИИ-агентов, ощущают её непосредственно. Отношение программистов к ИИ-программированию прошло через первоначальный шок и отрезвление: на практике, в производственном

применении, они постепенно осознали его ограниченность. Например, в одной из записей в блоге Кэла Ньюпорта (профессора информатики Джорджтаунского университета)^[3] рассказывается об опытном программисте из Кремниевой долины и о переменах, произошедших с ним за эти полгода, — о разработчике, который в начале года после программирования с помощью Claude Code поднял производительность в несколько раз и восклицал «really, really good», но, пережив два производственных сбоя из-за кода, написанного ИИ, и едва не потеряв работу, разочаровался в ИИ-программировании, снова взялся писать ключевой код вручную, а ИИ использует лишь для тестов и скриптов. По мнению этого разработчика, «старинное» программирование хоть и медленнее, зато надёжно и качественно. Автор отмечает, что случай этот не единичный: немало инженеров отрасли высказываются похожим образом.

В чём же проблема ИИ-программирования? Один программист в своём блоге^[4] делится впечатлениями от полугодовой проверки кода, написанного ИИ. По его мнению, ИИ-агент подобен очень способному новому сотруднику: код выглядит хорошо написанным, но из-за отсутствия знаний о предметной области он всегда недостаточно продумывает различные ограничения, граничные и аварийные сценарии, да и легко допускает проблемы с безопасностью. Об этом же он приводит работу «Угасание ограничений: хрупкость LLM-агентов при генерации серверного кода»^[5], в которой также разбирается этот вопрос:

Наше исследование выявило явление «угасания ограничений»: по мере роста структурных требований производительность агентов заметно падает. При переходе от базовых задач к задачам с полностью заданными требованиями у лучших конфигураций доля пройденных проверок (assert) снижалась в среднем на 30 процентных пунктов, а у некоторых слабых конфигураций падала почти до нуля. Анализ чувствительности к фреймворкам выявил значительные различия в производительности: агенты хорошо работают в минималистичных и явных фреймворках (таких как Flask), но в средах, сильно зависящих от соглашений (таких как FastAPI, Django), средняя производительность резко падает. Наконец, анализ ошибок показал, что главный источник этих проблем — дефекты уровня данных (например, неверное построение запросов и runtime-исключения ORM). Это исследование подчёркивает: одновременное выполнение функциональных и структурных требований остаётся для программирующих агентов ключевой и до сих пор нерешённой задачей.

Технический директор одной индийской компании в недавней статье также упомянул об этом^[6] и отметил, что код, написанный ИИ, легко даёт сбой в реальной производственной эксплуатации, поскольку «в больших системах код должен соответствовать архитектурным шаблонам, цепочкам зависимостей, бизнес-правилам, требованиям комплаенса, средствам безопасности и процессам релиза. Если сгенерированный ИИ код игнорирует эти ограничения, центр тяжести работы смещается с разработки на последующие исправления и доработки, что ведёт к удлинению циклов ревью кода, недостаточному покрытию тестами, обнаружению угроз безопасности, сбоям интеграции и дефектам в производственной среде». У кого есть реальный опыт ИИ-программирования, тот понимает, что эти проблемы действительно широко распространены.

ИИ пишет код быстро, но из-за перечисленных выше проблем разработчики вынуждены тщательно проверять сгенерированный код, прежде чем включить его в репозиторий. Отсюда вытекают две проблемы. Во-первых, скорость проверки кода заметно ниже скорости его генерации, и прирост эффективности от ИИ сильно обесценивается. Во-вторых, у молодых программистов нет возможности писать код своими руками, их навыки не растут; когда старшее поколение программистов сократят или оно выйдет на пенсию, скорость проверки кода в конечном счёте ещё больше упадёт, а то и вовсе никому будет заметить ошибки.

Более того, некоторые компании возлагают на ИИ-программирование большие надежды и резко сокращают сроки разработки, так что у программистов вообще не остаётся времени тщательно проверять код. Для разработчика предприятие не его собственное: начальство требует быстро — значит, проверка будет поверхностной или её не будет вовсе, и код сразу пойдёт в слияние. Если после запуска возникнут проблемы — ну, можно и уйти. Усталость от проверок или их недостаточность — итог один: в код, попадающий в промышленную эксплуатацию, закрадывается куча багов и «дурных запашков», а люди, отвечающие за сопровождение этих программ, на самом деле плохо знакомы с кодом. Весь этот технический долг и порождает так называемый «разрыв в сопровождаемости» (The Maintainability Gap)^[7].

Программирование с помощью ИИ продолжает бурно развиваться, и объём кода, производимый самими активными разработчиками, достиг исторического максимума. Однако анализ данных об изменениях кода за четыре года показывает, что показатели сопровождаемости кода падают: вызовы функций между файлами (свидетельство повторного использования кода) сократились на 35 %; по сравнению с уровнем 2022 года перенос строк кода при рефакторинге сократился на 70 %, а работа по сопровождению давно унаследованного кода — на 74 %. В то же время мы наблюдаем рост ряда тревожных явлений: «копирование/вставка» в рамках одного коммита выросли на 41

%, дублирование блоков кода — на 81 %, маскирующие ошибки конструкции кода — на 47 %, а текучесть кода (churn) в течение двух недель — на 15 %. Хотя выпуск кода действительно сильно вырос, накопленный вместе с ним технический долг столь же нельзя игнорировать; этот долг сосредоточен в основном у тех разработчиков, которые ещё не осознали, что некоторые приёмы кодирования подрывают долгосрочную сопровождаемость кодовой базы.

Что касается технического долга, то в работе «Debt Behind the AI Boom»^[8] (по данным крупномасштабного исследования) говорится: «Самыми распространёнными типами проблем оказались "дурные запахи" кода — на них приходится 89,3 %»; «в коммитах, созданных различными ИИ-ассистентами программирования, более 15 % вносили по меньшей мере одну проблему. Что важнее, 22,7 % проблем, внесённых ИИ, сохраняются и в последней версии репозитория». «Эти находки говорят о том, что сгенерированный ИИ код может приносить реальным программным проектам долгосрочные издержки сопровождения, а также подчёркивают необходимость усиления контроля качества при разработке с помощью ИИ».

Figure 3

Из работы «Debt Behind the AI Boom»

А в работе «Faster Code, Deeper Debt?»^[9] также показано: LLM нередко усугубляют традиционные формы технического долга — особенно в части кода, проектирования и документации — и одновременно порождают новые виды долга, специфические для LLM. «Примечательно, что мы выявили "долг быстрой интеграции": ради скорости принося в жертву качество, быстро сгенерированный код вызывает цепную реакцию, порождая долг управления и увеличивая долгосрочные издержки сопровождения. К другим новым категориям долга относятся долг подсказок (prompt), этический долг, долг данных и долг происхождения, — они отражают уникальные вызовы, которые несёт применение LLM».

ИИ-программирование не только ведёт к накоплению технического долга, но и вводит другие виды долга. Учёный-информатик Маргарет-Энн Стори в своей работе «From Technical Debt to Cognitive and Intent Debt»^[10] отмечает: помимо технического долга, по мере того как скорость генерации кода ИИ превышает скорость понимания его командой, накапливаются две часто недооцениваемые формы долга, и выделяет, во-первых, «когнитивный долг» (cognitive debt) — утрату общего понимания внутри команды, и, во-вторых, «долг намерения» (intent debt) — отсутствие внешне выраженного логического обоснования, необходимого разработчикам и ИИ-агентам для безопасной работы с кодом. Попросту говоря, повсеместное использование ИИ-программирования может привести к тому, что вся команда перестанет понимать, чем она занимается. Пока внимание отрасли сосредоточено на техническом долге, а эти более глубокие проблемы ещё не проявились в полной мере.

Думаю, у читателя к этому месту уже сложился ответ. Во многих своих ответах (например: «Как вы думаете, действительно ли ИИ-программирование в будущем заменит программистов?») и «Как относиться к прогнозу главы направления ИИ в Microsoft о том, что в течение ближайших 18 месяцев большинство белых воротничков будут полностью заменены ИИ?») я не раз повторял одну мысль: ИИ глубоко изменит способы работы белых воротничков, но ИИ не может вытеснить всех программистов и тем более не может оставить без работы всех белых воротничков. Реальность подтвердила правоту моей точки зрения.

По мере того как миф о замене ИИ всех белых воротничков развевается, на рынке капитала уже появляются признаки лопающегося пузыря. Ранее, исходя из предположения о 30-процентном приросте эффективности в разработке ПО, я пришёл к выводу, что пузырь ИИ действительно существует и масштабы его больше, чем у пузыря интернет-компаний (см.: «Насколько велик пузырь ИИ? Превысит ли кризис пузыря ИИ кризис 2008 года?»).

А недавние речи генеральных директоров компаний, делающих большие модели ИИ, об уничтожении человечества ИИ — не более чем последнее безумие перед гибелью.

Ссылки

- ^The AI revolution in software development <https://www.mckinsey.com/capabilities/tech-and-ai/our-insights/the-ai-revolution-in-software-development>
- ^Writing Code vs. Shipping Code: Productivity Effects Across Generations of AI Coding Tools. <https://download.ssrn.com/2026/9/6/6843118.pdf?response-content-disposition=inline&X-Amz-Security-Token=IQoJb3JpZ2luX2VjElr%2F%2F%2F%2F%2F%2F%2F%2F%2F%2F%2FwEaCXVzLWVhc3QtMSJHMEUCIA4KlxaTLIbYG8l%2B4GGU>
- ^On AI Coding and Its Discontents <https://calnewport.com/on-ai-coding-and-its-discontents/>

- ^I Spent 6 Months Reviewing AI Code. Here's the Ugly Truth https://medium.com/@haseeb_sohail/i-spent-6-months-reviewing-ai-code-heres-the-ugly-truth-b81fe33e0b25
- ^Constraint Decay: The Fragility of LLM Agents in Backend Code Generation <https://arxiv.org/pdf/2605.06445>
- ^The Productivity Illusion at the Heart of Enterprise AI Coding <https://cacm.acm.org/blogcacm/the-productivity-illusion-at-the-heart-of-enterprise-ai-coding/>
- ^The Maintainability Gap: AI Code Quality in 2026
https://www.gitclear.com/the_ai_code_quality_maintainability_gap
- ^Debt Behind the AI Boom: A Large-Scale Empirical Study of AI-Generated Code in the Wild
<https://arxiv.org/abs/2603.28592>
- ^Faster Code, Deeper Debt? A Multivocal Literature Review on Technical Debt and Its Early Signs in LLM-Assisted Software Development <https://arxiv.org/abs/2606.14796>
- ^From Technical Debt to Cognitive and Intent Debt: Rethinking Software Health in the Age of AI
<https://arxiv.org/abs/2603.22106>

<https://arxiv.org/abs/2603.28592>

[9] Faster Code, Deeper Debt? A Multivocal Literature Review on Technical Debt and Its Early Signs in LLM-Assisted Software Development — arXiv

中文原文: Faster Code, Deeper Debt? A Multivocal Literature Review on Technical Debt and Its Early Signs in LLM-Assisted Software Development

<https://arxiv.org/abs/2606.14796>

[10] From Technical Debt to Cognitive and Intent Debt: Rethinking Software Health in the Age of AI — arXiv

中文原文: From Technical Debt to Cognitive and Intent Debt: Rethinking Software Health in the Age of AI

<https://arxiv.org/abs/2603.22106>

Эта статья изначально была опубликована на Zhihu (на китайском языке): <https://zhuanlan.zhihu.com/p/2084561490930045240> Автор: 最最遥远的路 · Профиль на Zhihu: <https://www.zhihu.com/people/zui-zui-yao-yuan-de-lu-32/posts>

本文原发表于知乎专栏/回答 (中文) : <https://zhuanlan.zhihu.com/p/2084561490930045240>; 作者: 最最遥远的路: <https://www.zhihu.com/people/zui-zui-yao-yuan-de-lu-32/posts>